

Estructuras de Control, Ciclos y Condicionales ($\mathcal{C}^3$)

Aproximación y error · Representación numérica · IEEE 754 · Suma de series

Carlos Pinilla

`pinilla.carlos@correounivalle.edu.co`

- 1 Control del flujo: condicionales, ciclos y criterios de parada.
- 2 Aproximación y error: fuentes, error absoluto/relativo, dato vs. computacional.
- 3 Forward/backward error, condicionamiento y estabilidad.
- 4 Representación de números: bits, IEEE 754 y punto flotante.
- 5 Precisión de máquina y cancelación.
- 6 Aplicación: suma de series para calcular $\sin(x)$.
- 7 Problema físico adicional: decaimiento radiactivo con Python.

De FORTRAN a Python: misma física, otra herramienta

Semana 1 – FORTRAN

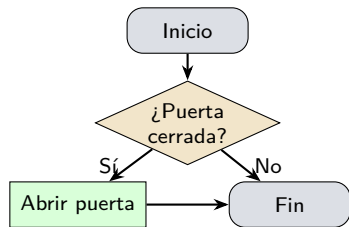
- Variables y tipos básicos.
- Entrada/salida.
- Primeros algoritmos secuenciales.
- Problema físico: decaimiento radiactivo.

Semana 2 – Python

- Condicionales y ciclos.
- Criterios de parada.
- Arreglos y visualización.
- Precisión y representación numérica.

La programación cambia; el razonamiento físico permanece.

Pensar antes de programar



Idea central

Antes de escribir código, es útil identificar:

- el estado inicial;
- las decisiones;
- las acciones;
- las condiciones de salida.

En física computacional

Un diagrama de flujo puede evitar errores lógicos antes de que aparezcan como errores de código.

Condicionales

Los condicionales son pruebas que determinan si una comparación es **verdadera** o **falsa**. En Python se implementan con `if`, `elif` y `else`.

Operador	Python	Significado
<code>==</code>	<code>a == b</code>	igualdad
<code>!=</code>	<code>a != b</code>	diferencia
<code>></code> , <code><</code>	<code>a > b</code>	mayor / menor
<code>>=</code> , <code><=</code>	<code>a >= b</code>	mayor o igual / menor o igual
<code>and</code> , <code>or</code> , <code>not</code>	<code>a>0 and b>0</code>	lógica booleana

Ejemplo físico

Verificar que una masa, una distancia o un paso temporal sean positivos antes de iniciar una simulación.

Condicionales en Python

```
1 temperatura = 150.0
2
3 if temperatura > 100:
4     print("Ebullicion")
5 elif temperatura == 0:
6     print("Congelacion")
7 else:
8     print("Liquido")
```

Cuidado

En Python la indentación define qué instrucciones pertenecen a cada bloque. Esto no es estilo: es parte de la sintaxis del lenguaje.

Ciclos: for y while

Ciclo for: número de repeticiones conocido

```
1 for i in range(5):  
2     print(i)
```

Ciclo while: repetir hasta cumplir una condición

```
1 x = 5  
2 while x > 0:  
3     print(x)  
4     x -= 1
```

Uso en física computacional

for: recorrer pasos temporales o datos.
alcanzar una tolerancia.

while: iterar hasta convergencia o hasta

Criterios de parada

Un criterio de parada determina **cuándo detener un algoritmo iterativo**.

Criterios comunes

- Tolerancia absoluta: $|x_{n+1} - x_n| < \varepsilon$.
- Tolerancia relativa: $|\Delta x|/|x| < \varepsilon$.
- Máximo número de iteraciones.

Balance

- Muy estricto: cálculo costoso.
- Muy laxo: resultado impreciso.
- Sin máximo de iteraciones: posible ciclo infinito.

Estas tolerancias son, precisamente, el mismo lenguaje de error relativo/absoluto que formalizamos a continuación – y que retomamos más adelante al hablar de condicionamiento.

¿De dónde viene el error?

Todo resultado numérico es una **aproximación**. La pregunta no es si hay error, sino de dónde viene y qué tan grande es.

Antes de computar

- Modelado: simplificar la realidad.
- Mediciones empíricas.
- Cálculos previos ya aproximados.

Durante el cómputo

- Truncamiento/discretización – aproximación *matemática*.
- Redondeo – aproximación *aritmética*.

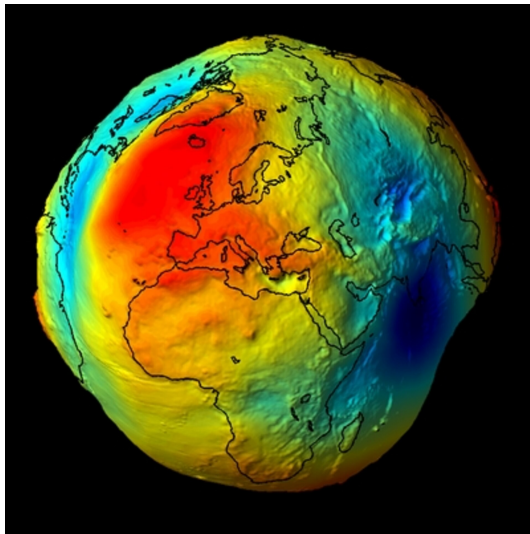
“He aprendido que, en la descripción de la Naturaleza, hay que tolerar aproximaciones, y que trabajar con aproximaciones puede ser interesante y a veces hermoso.” – P. A. M. Dirac

Ejemplo: aproximaciones apiladas

Calcular el área de la Tierra con $A = 4\pi r^2$ involucra **varias** aproximaciones independientes:

- la Tierra se modela como una esfera, idealizando su forma real;
- el valor de r viene de una medición empírica;
- el valor de π requiere truncar un proceso infinito;
- los datos de entrada y los resultados intermedios se redondean.

Ejemplo: aproximaciones apiladas



Definiciones

error absoluto = valor aproximado – valor real

$$\text{error relativo} = \frac{\text{error absoluto}}{\text{valor real}}$$

- Equivalentemente, valor aprox. = (valor real) $\times$ (1 + error relativo).
- Puede expresarse como porcentaje: error % = error relativo $\times$ 100.
- El valor real casi nunca se conoce exactamente $\rightarrow$ el error se *estima* o se *acota*, no se calcula.
- En la práctica, el error relativo suele tomarse respecto al valor **aproximado** (no al real, que es desconocido).

Error de dato vs. error computacional

Al evaluar $\hat{f}(\hat{x})$ en vez del ideal $f(x)$, el error total se descompone en dos partes independientes:

$$\underbrace{\hat{f}(\hat{x}) - f(x)}_{\text{error total}} = \underbrace{\hat{f}(\hat{x}) - f(\hat{x})}_{\text{error computacional}} + \underbrace{f(\hat{x}) - f(x)}_{\text{error de dato propagado}}$$

Clave

El algoritmo **no** tiene ningún efecto sobre el error de dato propagado – es responsabilidad de la calidad de la entrada, no del código.

Ejemplo: error de dato vs. error computacional

Aproximar $\sin(\pi/8)$ “a mano”, sin calculadora: usamos $\hat{x} = 3/8$ en vez de $x = \pi/8$, y $\hat{f}(x) = x$ (primer término de Taylor) en vez de $f(x) = \sin(x)$.

$$y = \sin(\pi/8) \approx 0,3827 \qquad \hat{y} = \hat{f}(\hat{x}) = 3/8 = 0,3750$$

$$\text{error computacional} = \hat{f}(\hat{x}) - f(\hat{x}) \approx +0,0087$$

$$\text{error de dato propagado} = f(\hat{x}) - f(x) \approx -0,0164$$

$$\text{error total} = \hat{y} - y \approx -0,0077$$

Truncamiento y redondeo

- **Error de truncamiento:** diferencia entre el resultado exacto y el que da un algoritmo con aritmética exacta – viene de reemplazar un proceso infinito por uno finito (serie truncada, derivada discretizada, iteración detenida antes de converger).
- **Error de redondeo:** diferencia entre el resultado de un algoritmo con aritmética exacta y el mismo algoritmo con aritmética de precisión limitada.

Regla práctica

Error computacional = truncamiento + redondeo. Casi siempre **uno de los dos domina** al otro.

Ejemplo clásico: la diferencia finita $f'(x) \approx (f(x+h) - f(x))/h$, h grande favorece el truncamiento y h chico favorece el redondeo – existe un h óptimo.

Forward y backward error

- **Forward error:** $\Delta y = \hat{y} - y$ – qué tan lejos quedó el resultado.
- **Backward error:** $\Delta x = \hat{x} - x$ – qué tan lejos tendría que estar la *entrada* para que el resultado obtenido fuera *exacto*.

Idea

El backward error suele ser más fácil de estimar que el forward error, y es la base del análisis de estabilidad: la solución aproximada es la solución *exacta* de un problema ligeramente distinto (el problema “vecino”).

Ejemplo: forward y backward error

Como aproximación a $y = \sqrt{2}$, usamos $\hat{y} = 1,4$.

$$\text{forward: } |\Delta y| = |\hat{y} - y| \approx 0,0142 \quad (\text{rel.} \approx 1 \%)$$

$$\text{backward: } |\Delta x| = |\hat{x} - x| = |1,96 - 2| = 0,04 \quad (\text{rel.} \approx 2 \%, \text{ con } \hat{x} = \hat{y}^2)$$

La razón entre el error forward relativo y el backward relativo es tan importante que le daremos nombre propio: el **número de condición**.

Backward error analysis

Idea: la solución aproximada es la solución exacta de un problema *modificado*. Si ese backward error es menor que la incertidumbre que ya traía la entrada, ¡la aproximación es tan buena como el problema lo permite!

Ejemplo: $\cos(x)$ truncado a dos términos de Taylor

$\hat{f}(x) = 1 - x^2/2$, evaluado en $x = 1$:

$$\hat{y} = 0,5, \quad \hat{x} = \arccos(\hat{y}) \approx 1,0472$$

$$\text{forward: } \Delta y \approx -0,0403 \quad \text{backward: } \Delta x \approx +0,0472$$

Se reproduce en vivo en el notebook del docente.

Condicionamiento

Problema bien puesto (well-posed)

Existe solución, es única, y depende *continuamente* de los datos. Si no, el problema es **mal puesto**.

Número de condición

$$\text{cond}(f) = \frac{|\text{cambio relativo en la salida}|}{|\text{cambio relativo en la entrada}|} = \frac{|\Delta y/y|}{|\Delta x/x|}$$

- **Bien condicionado:** $\text{cond} \approx 1$ – la solución es insensible a perturbaciones pequeñas.
- **Mal condicionado:** $\text{cond} \gg 1$ – un cambio relativo pequeño en la entrada puede producir un cambio relativo enorme en la salida.
- Condicionamiento es propiedad del **problema**, no del algoritmo.

Estabilidad y exactitud

Estabilidad

Un algoritmo es **estable** si el resultado que produce es relativamente insensible a perturbaciones *durante* el cómputo – equivalente, por análisis de error hacia atrás, a que sea la solución exacta de un problema cercano.

Exactitud

La exactitud (cercanía al resultado real) depende de **ambos**: condicionamiento del problema y estabilidad del algoritmo.

Resumen

Algoritmo estable + problema bien condicionado $\Rightarrow$ resultado exacto. Cualquier otra combinación puede fallar – incluyendo un algoritmo estable sobre un problema mal condicionado.

Ejemplo: número de condición

$\tan(x)$ **cerca de** $\pi/2$:

$$\tan(1,57079) \approx 1,58 \times 10^5 \quad \tan(1,57078) \approx 6,12 \times 10^4$$

Un cambio relativo minúsculo en x produce un cambio relativo un *cuarto de millón de veces mayor* en la salida: $\text{cond} \approx 2,5 \times 10^5$ – mal condicionado cerca de la asíntota.

Contraste: $\sqrt{x}$ y x^2

$\text{cond}(\sqrt{x}) = \frac{1}{2}$ y $\text{cond}(x^2) = 2$ – recíprocos entre sí, porque son operaciones inversas. Ambas están bien condicionadas.

Se calcula en vivo con la función `cond_numerico` del notebook del docente.

Representación de números en computadores

- Los computadores son sistemas finitos: usan memoria limitada para representar números.
- La unidad mínima de información es el **bit**: 0 o 1.
- Un **byte** contiene 8 bits.
- Con N bits, un entero sin signo puede representar

$$0, 1, 2, \dots, 2^N - 1.$$

Ejemplos

- 8 bits: 0 a 255.
- 32 bits sin signo: 0 a $2^{32} - 1 \approx 4,29 \times 10^9$.
- 64 bits sin signo: 0 a $2^{64} - 1 \approx 1,84 \times 10^{19}$.

Memoria, palabra y escalas físicas

- La **longitud de palabra** indica cuántos bits se usan para almacenar o procesar una cantidad.
- En computadores modernos son comunes arquitecturas de 64 bits.
- Los límites de representación pueden ser suficientes para muchas tareas, pero no para todas las escalas físicas.

Comparación de escala

La razón entre una escala cosmológica y una escala subatómica puede ser enorme. Por ejemplo, la relación entre el tamaño del universo observable y el tamaño de un protón es del orden de 10^{41} . Esto ilustra por qué no basta con “usar números grandes”: hay que entender el formato numérico.

Enteros en Python y en NumPy

Python puro

Los enteros pueden crecer tanto como permita la memoria disponible.

$$2^{1000}$$

puede calcularse exactamente.

NumPy

Los tipos como `int32` o `int64` tienen tamaño fijo.

$$\text{int64} : -2^{63} \text{ a } 2^{63} - 1$$

Aquí sí puede aparecer **overflow**.

Mensaje

No todos los números en Python se comportan igual: el tipo de dato importa.

Números en punto flotante

Un sistema de punto flotante queda definido por cuatro enteros: la **base** β , la **precisión** p , y el **rango de exponente** $[L, U]$. Un número real se representa como

$$x = \pm \left(d_0 + \frac{d_1}{\beta} + \cdots + \frac{d_{p-1}}{\beta^{p-1}} \right) \beta^E, \quad 0 \leq d_i \leq \beta - 1, \quad L \leq E \leq U.$$

Notación equivalente y más compacta

$$x = (-1)^s \times M \times \beta^E$$

con s el signo, M la mantisa (los d_i empaquetados) y E el exponente.

Idea clave

El punto flotante representa números muy grandes y muy pequeños, pero con un número **limitado** de dígitos significativos – el sistema es finito y discreto, no continuo.

Sistemas típicos de punto flotante

Sistema	β	p	L	U
IEEE precisión media	2	11	-14	15
IEEE precisión simple	2	24	-126	127
IEEE precisión doble	2	53	-1022	1023
IEEE precisión cuádruple	2	113	-16382	16383
Calculadora HP	10	12	-499	499
IBM mainframe (histórico)	16	6	-64	63

- El par (β, p, L, U) define **cualquier** sistema – IEEE 754 (binario) es solo el que domina hoy.
- Computadores modernos usan aritmética binaria ($\beta = 2$); IEEE 754 es casi universal.

Un sistema de punto flotante es **normalizado** si el primer dígito d_0 siempre es distinto de cero (excepto para representar el cero mismo). En un sistema normalizado, la mantisa satisface

$$1 \leq M < \beta.$$

¿Por qué normalizar?

- Cada número tiene una **única** representación.
- No se desperdician dígitos en ceros a la izquierda.
- En binario ($\beta = 2$), d_0 siempre es 1 – por lo que **no hace falta almacenarlo**. Así es como IEEE 754 obtiene un dígito “gratis” en la mantisa (el “1.” implícito de $1.f$).

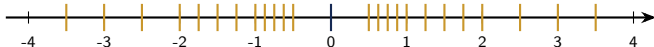
Propiedades de un sistema de punto flotante

- El sistema es **finito y discreto**: solo existe una cantidad limitada de números representables.
- Cantidad total de números normalizados: $2(\beta - 1)\beta^{p-1}(U - L + 1) + 1$.
- Menor número positivo normalizado: $\text{UFL} = \beta^L$.
- Mayor número representable: $\text{OFL} = \beta^{U+1}(1 - \beta^{-p})$.

La observación más importante

Los números de punto flotante están **igualmente espaciados solo entre potencias sucesivas de β** . La separación entre números representables *crece* con la magnitud – por eso no todos los reales son “machine numbers”.

Ejemplo: sistema de punto flotante de juguete

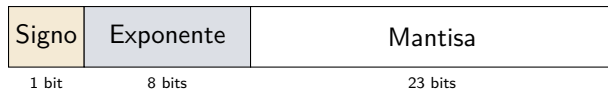


Sistema con $\beta = 2$, $p = 3$, $L = -1$, $U = 1$ – 25 números representables (marcas doradas) en total.

- $\text{OFL} = (1,11)_2 \times 2^1 = 3,5$ $\text{UFL} = (1,00)_2 \times 2^{-1} = 0,5$
- Cerca de 0 los números están muy juntos; cerca de 3.5 están mucho más separados.

A suficiente magnificación, todo sistema de punto flotante – incluido `float64` – se ve así de granoso y desigualmente espaciado. Se visualiza en vivo en el notebook del docente.

IEEE 754: estructura de un float32



- Signo: 0 positivo, 1 negativo.
- Exponente: almacenado con sesgo. Para float32, bias = 127.
- Mantisa: fracción binaria después del punto.

$$x = (-1)^s \times 1.f \times 2^{e-127} \quad (\text{números normalizados}).$$

Aquí e es el exponente almacenado (con sesgo, 0 a 255 en float32); el exponente E de la definición general $E = e - 127$ – por eso aparece como 2^{e-127} y no como 2^e .

Precisión simple y doble

Formato	Total	Signo	Exponente	Fracción
Precisión simple	32 bits	1	8	23
Precisión doble	64 bits	1	11	52

En Python

En CPython, el tipo integrado `float` usa normalmente IEEE 754 de doble precisión (binary64). Con NumPy podemos pedir explícitamente `np.float32` o `np.float64`.

No confundir

Rango indica qué tan grande o pequeño puede ser un número. **Precisión** indica cuántos dígitos significativos podemos confiar.

Ejemplo: codificar 5.75 en IEEE 754

- 1 $5,75_{10} = 101,11_2$.
- 2 Normalización:
 $101,11_2 = 1,0111_2 \times 2^2$.
- 3 Exponente sesgado:
 $2 + 127 = 129 = 10000001_2$.
- 4 Signo: $s = 0$.
- 5 Fracción: 011100000000000000000000.

0 10000001 011100000000000000000000

Verificación en Python

```
1 import struct
2
3 b = struct.pack('>f', 5.75)
4 bits = ''.join(f'{x:08b}' for x in b)
5 print(bits)
```

Casos especiales en IEEE 754

Caso	Condición (simple)	Valor
Normal	$0 < e < 255$	$(-1)^s 2^{e-127} 1.f$
Subnormal	$e = 0, f \neq 0$	$(-1)^s 2^{-126} 0.f$
Cero	$e = 0, f = 0$	± 0
$+\infty$	$s = 0, e = 255, f = 0$	$+\infty$
$-\infty$	$s = 1, e = 255, f = 0$	$-\infty$
NaN	$e = 255, f \neq 0$	No es un número

Importancia

Estos casos aparecen en cálculos reales: divisiones por cero, raíces de números negativos, overflow o resultados indefinidos.

Rango de números en punto flotante

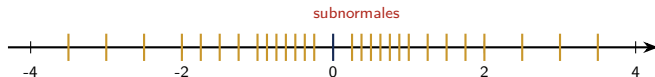
Formato	Mínimo positivo subnormal	Máximo finito
float32	$\sim 1,4 \times 10^{-45}$	$\sim 3,4 \times 10^{38}$
float64	$\sim 4,9 \times 10^{-324}$	$\sim 1,8 \times 10^{308}$

- **Overflow:** el resultado excede el máximo finito.
- **Underflow:** un número positivo cae por debajo del mínimo representable.
- Entre cero y el mínimo normal existen números **subnormales**, con precisión reducida.

¿Cuál es peor?

Overflow es más grave que underflow: no existe una buena aproximación para un número gigante, pero **0 es una aproximación razonable** de un número diminuto. Por eso el overflow suele ser fatal, mientras que el underflow a veces se pone en 0 silenciosamente.

Subnormales y underflow gradual



- La normalización ($d_0 \neq 0$) deja un **hueco** alrededor de cero – entre 0 y el UFL normal no hay ningún número representable.
- Los **subnormales** rellenan ese hueco: se permite $d_0 = 0$ solo cuando el exponente está en su valor mínimo.
- Resultado: **underflow gradual** en vez de un salto abrupto de UFL a 0.

Por eso `sys.float_info.min` (UFL normal) no es el positivo más pequeño representable: por debajo de él todavía hay subnormales, hasta $5e-324$ en `float64`.

Explorar límites en Python

```
1 import sys
2 import numpy as np
3
4 print(sys.float_info.max)
5 print(sys.float_info.min)
6 print(np.finfo(np.float64).eps)
7 print(np.finfo(np.float32).eps)
```

Experimentos en el notebook

- ¿Qué pasa con $1e308 * 10$?
- ¿Qué pasa con $5e-324 / 2$?
- ¿Qué diferencia hay entre float64 y float32?

Reglas de redondeo

Si un número real x no es exactamente representable, se aproxima por un número de punto flotante “cercano” $\text{fl}(x)$ – a ese proceso se le llama **redondeo**. Hay dos reglas comunes:

Chop (truncar)

Se descartan los dígitos después del $(p - 1)$ -ésimo – equivale a redondear siempre hacia cero.

$$\varepsilon_{\text{mach}} = \beta^{1-p}.$$

Round to nearest (redondear al más cercano)

$\text{fl}(x)$ es el número representable más próximo a x (con “round to even” en caso de empate). Es la regla **por defecto en IEEE** y la más exacta. $\varepsilon_{\text{mach}} = \frac{1}{2}\beta^{1-p}$.

Definición práctica

La precisión de máquina (o *unit roundoff*) ε_m es la separación entre 1 y el siguiente número representable mayor que 1.

$$1 + \varepsilon_m > 1, \quad \left| \frac{\text{fl}(x) - x}{x} \right| \leq \varepsilon_{\text{mach}}.$$

Sistema de juguete ($\beta = 2, p = 3$):

- chop: $\varepsilon_m = (0,01)_2 = 0,25$
- nearest: $\varepsilon_m = (0,001)_2 = 0,125$

IEEE simple, doble y cuádruple tienen, respectivamente, unos 7, 16 y 36 dígitos decimales de precisión.

Sistemas IEEE:

- Simple: $\varepsilon_m \approx 1,19 \times 10^{-7}$
- Doble: $\varepsilon_m \approx 2,22 \times 10^{-16}$

Precisión de máquina vs. UFL: no confundir

Aunque los dos son “números chicos”, miden cosas distintas:

- $\varepsilon_{\text{mach}}$ está determinado por el número de dígitos de la **mantisa** – mide **precisión** (espaciado relativo entre números).
- UFL está determinado por el rango del **exponente** – mide **rango** (qué tan chico puede ser un número antes de dejar de ser representable).

En todo sistema de punto flotante práctico

$$0 < \text{UFL} < \varepsilon_{\text{mach}} < \text{OFL}.$$

Experimento: medir ε_m

```
1 eps = 1.0
2
3 while 1.0 + eps != 1.0:
4     eps_anterior = eps
5     eps = eps / 2.0
6
7 print(eps_anterior)
```

Nota: `numpy.finfo(float64).eps` reporta 2^{-52} , la fórmula de chop, aunque IEEE en realidad redondea a nearest (que daría 2^{-53}). “Machine epsilon” en NumPy se define como un ULP (la distancia de 1.0 al siguiente flotante), no como el error máximo real de redondeo.

Redondeo en la aritmética de punto flotante

Cada operación aritmética en punto flotante puede introducir su propio error de redondeo:

$$\text{fl}(x) = x(1 + \delta), \quad |\delta| \leq u.$$

- **Suma/resta:** para igualar exponentes se desplaza la mantisa – puede perderse parte (o todo) el número más chico.
- **Multipliación:** el producto de dos mantisas de p dígitos puede tener hasta $2p$ dígitos – no siempre representable.
- **División:** el cociente puede no terminar (p. ej. $1/10$ en binario).

Estas tres fuentes de redondeo son la parte “computacional” del error dentro de una sola operación; la siguiente diapositiva muestra qué pasa cuando se acumulan a través de una resta.

Cancelación

```
1 resultado = (1e16 + 1) - 1e16
2 print(resultado)
```

Matemáticamente

$$(10^{16} + 1) - 10^{16} = 1.$$

Computacionalmente

Cerca de 10^{16} , la separación entre números float64 representables es mayor que 1. Por eso el incremento puede perderse antes de hacer la resta.

La cancelación pierde dígitos más significativos (al restar cantidades casi iguales), a diferencia del redondeo, que pierde los menos significativos. En el notebook del docente se ve con consecuencias físicas reales: la energía total del átomo de helio (144 % de cambio en el total a partir de solo 4.6 % en los componentes), la fórmula cuadrática con $b^2 \gg 4ac$, la desviación estándar de una pasada vs. dos pasadas, la serie armónica que “converge” en float32, y la no-asociatividad de la suma en punto flotante.

Aplicación: suma de series

Un problema clásico en computación numérica es evaluar funciones mediante series:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Objetivo

Calcular $\sin(x)$ con una tolerancia prescrita usando un algoritmo finito.

Dificultad

La serie es exacta, pero el programa no suma infinitos términos. Además, potencias y factoriales pueden ser costosos o producir números intermedios muy grandes.

Algoritmo ingenuo vs. algoritmo recursivo

Forma ingenua

$$\sum_{n=0}^N (-1)^n \frac{x^{2n+1}}{(2n+1)!}$$

- Calcula potencias.
- Calcula factoriales.
- Puede ser ineficiente.

Forma recursiva

$$t_0 = x, \quad t_n = -t_{n-1} \frac{x^2}{(2n)(2n+1)}$$

- Reutiliza el término anterior.
- Evita factoriales explícitos.
- Es más estable y eficiente.

Criterio de parada para la serie

Sea $S_n = \sum_{k=0}^n t_k$. Una opción práctica es detener la suma cuando el último término aporta muy poco:

$$|t_n| < \text{tol} \cdot \max(1, |S_n|).$$

- Evita dividir por S_n cuando $\sin(x)$ está cerca de cero.
- Combina una escala absoluta y una escala relativa.
- Siempre debe acompañarse de un máximo número de iteraciones.

Pseudocódigo para $\sin(x)$

```
1 term = x
2 suma = x
3 n = 0
4
5 while abs(term) > tol * max(1.0, abs(suma)):
6     n = n + 1
7     term = -term * x**2 / ((2*n) * (2*n + 1))
8     suma = suma + term
```

Qué se debe reportar

- Valor calculado de $\sin(x)$.
- Número de términos usados.
- Error relativo frente a `math.sin(x)`.

Argumentos grandes: reducción del argumento

La serie de Taylor alrededor de cero converge para todo x , pero puede ser muy ineficiente o numéricamente problemática para $|x|$ grande.

Identidad trigonométrica

$$\sin(x + 2\pi m) = \sin(x), \quad m \in \mathbb{Z}.$$

Idea computacional

Antes de sumar la serie, podemos reducir el argumento a un intervalo centrado alrededor de cero, por ejemplo con

$$x_{\text{red}} = \text{remainder}(x, 2\pi).$$

Taller 2: resultados esperados

- 1 Implementar la suma recursiva de la serie de $\sin(x)$ en Python.
- 2 Comparar con `math.sin(x)`.
- 3 Construir una tabla con x , número de iteraciones, suma y error relativo.
- 4 Explorar valores pequeños y valores grandes de x .
- 5 Usar reducción de argumento para mejorar el comportamiento del algoritmo.
- 6 Incluir al final una celda de verificación con 2-3 casos conocidos que impriman PASS/FAIL (política de tareas del curso).

Pregunta central

¿Por qué dos algoritmos matemáticamente equivalentes pueden comportarse de manera diferente en un computador?

Problema físico adicional: decaimiento radiactivo

Durante la Semana 1 estudiamos el decaimiento radiactivo en FORTRAN. Ahora usaremos Python para analizar muchos tiempos y visualizar el resultado.

$$N(t) = N_0 e^{-\lambda t}, \quad \lambda = \frac{\ln 2}{t_{1/2}}.$$

Datos

$$N_0 = 10000, \quad t_{1/2} = 5 \text{ años.}$$

- Crear un arreglo de tiempos entre 0 y 30 años.
- Calcular $N(t)$ de forma vectorizada con NumPy.
- Graficar la evolución temporal.
- Identificar una, dos y tres vidas medias.

- Las estructuras de control permiten construir algoritmos; los criterios de parada conectan programación con precisión física.
- Todo error se descompone en error de dato (de la entrada) y error computacional (del algoritmo) – el algoritmo no puede arreglar el primero.
- El **condicionamiento** es del problema; la **estabilidad** es del algoritmo; la **exactitud** depende de ambos.
- IEEE 754 explica por qué el computador no representa todos los reales exactamente, y la precisión de máquina limita lo que podemos pedirle a un algoritmo.
- Redondeo pierde dígitos menos significativos; cancelación pierde dígitos más significativos – y es más traicionera.
- La suma de series muestra que el diseño del algoritmo importa tanto como la fórmula matemática.

Siguiente paso: notebook de Python.